



WEEK 5 - PYTHON PROGRAMMING

# Modules and NumPy

Imports, the standard library and a first look at NumPy

---

Session 1 - 120 minutes, including a break

Instructor: MinGi KYUNG

# Learning Goals & Agenda

---

- Import functions with import and from ... import.
- Explain the dot, NameError and the import styles.
- Package reusable functions in your own module.
- Apply math, random and datetime to real tasks.
- Compute with whole arrays in NumPy.

Review 8 min / Imports & math 20 min / Own modules 12 min / Break 10 min  
datetime 10 min / random 10 min / Labs 15 min / NumPy 30 min / Review & quiz 5 min

# Review: One Function, Two Programs

---

- `sum_to(n)` from Week 3 returns  $1 + 2 + \dots + n$ .
- Imagine a second program that needs the same function.
- Copying the code means fixing every bug twice.
- Today: give reusable code a home of its own.

```
def sum_to(n):  
    total = 0  
    for value in range(1, n + 1):  
        total += value  
    return total  
  
print(sum_to(10))    # 55
```

# What Is a Module?

---

- A module is a file containing Python definitions.
- The file name minus .py is the module name.
- Importing runs the module's top-level code once.
- Every program you have written was already a module.

```
# File: greetings.py
def hello(name):
    return "Hello, " + name + "!"
```

# import and the Dot

---

- `import math` loads the `math` module.
- The dot reaches a name inside it: `math.sqrt(2)`.
- Keeping the prefix makes each name's origin visible.
- Python caches a module after its first import.

```
import math

print(math.pi)           # 3.141592653589793
print(math.sqrt(2))      # 1.4142135623730951
print(math.floor(2.9))   # 2
```

# from module import name

---

- Import one name and drop the prefix.
- Several names can be listed, separated by commas.
- Renaming while importing is also possible.
- Only the names you import become visible.

```
from math import sqrt, pi

print(sqrt(16)) # 4.0
print(pi)       # 3.141592653589793
```

# Import Styles Compared

---

- `import module`: clearest origin, a little more typing.
- `from module import name`: short, best for a few names.
- `import module as m`: a shorter prefix, useful aliases.
- `from module import *` hides where names came from.

```
import math                # math.sqrt(2)
from math import sqrt      # sqrt(2)
import math as m           # m.sqrt(2)
from math import sqrt, pi  # sqrt(2), pi
```

# NameError: Reading the Traceback

---

- The traceback names the file and the failing line.
- NameError: name 'sqrt' is not defined.
- The cause here: math was never imported.
- Read the last line first, then fix and rerun.

```
print(sqrt(16))    <-- the failing line
```

```
Traceback (most recent call last):  
  File "demo.py", line 1, in <module>  
NameError: name 'sqrt' is not defined
```

# math Essentials

---

- Constants: pi and e.
- sqrt, floor and ceil for roots and rounding.
- factorial(n) and gcd(a, b) work on integers.
- hypot(a, b) is the diagonal of a right triangle.

```
import math

print(math.floor(2.5))    # 2
print(math.ceil(2.5))     # 3
print(math.factorial(5))  # 120
print(math.gcd(12, 18))   # 6
print(math.hypot(3, 4))   # 5.0
```

# Practice 1: Circle & Hypotenuse

---

- Open `math_practice.py`.
- Read a radius; print area and circumference.
- Round both results to two decimals.
- Then read two legs; print the hypotenuse. 5 minutes.

Assume numeric input. Use `math.pi` and `math.hypot`.

```
Radius: 3  
Area: 28.27  
Circumference: 18.85
```

```
First leg: 3  
Second leg: 4  
Hypotenuse: 5.0
```

# The Standard Library: Batteries Included

---

- Python ships with hundreds of tested modules.
- No installation and no download are needed.
- Today: math, random and datetime.
- Full reference: [docs.python.org/3/library](https://docs.python.org/3/library).

| Module     | Example use                             |
|------------|-----------------------------------------|
| math       | <code>sqrt, pi, factorial</code>        |
| random     | <code>randint, choice, sample</code>    |
| datetime   | <code>today, strftime, timedelta</code> |
| statistics | <code>mean, median</code>               |
| csv, json  | <code>files and data exchange</code>    |

# Your First Module: number\_tools.py

---

- Reusable code lives in its own .py file.
- average returns the mean of a list of numbers.
- The module defines tools; it does not talk to the user.
- Place it next to the program that imports it.

```
# File: number_tools.py
def average(numbers):
    total = sum(numbers)
    return total / len(numbers)

def spread(numbers):
    return max(numbers) - min(numbers)
```

# Importing Your Own File

---

- stats\_report.py imports the tools it needs.
- number\_tools.average(...) calls your own function.
- Both files stay in the same folder.
- After editing a module, run the program again.

```
# File: stats_report.py
import number_tools

scores = [88, 92, 79, 93]

print(number_tools.average(scores))
print(number_tools.spread(scores))
# 88.0
# 14
```

# Run or Import: `__name__ == "__main__"`

---

- Running a file directly sets `__name__` to `"__main__"`.
- Importing it sets `__name__` to the module name.
- The guard separates reusable tools from demo code.
- Imports stay silent; direct runs show the demo.

```
# added at the bottom of number_tools.py
if __name__ == "__main__":
    demo = [2, 4, 6, 8]
    print(average(demo))
# printed only when run directly
```

# Packages: Modules in Folders

---

- A package is a folder of related modules.
- `__init__.py` marks the folder as a package.
- Dotted names navigate:  
`package.module.function`.
- Packages return with pip and external libraries in Week 7.

```
course/  
  __init__.py  
  stats.py      -> course.stats  
  tools.py      -> course.tools  
  
import course.stats  
course.stats.average([1, 2, 3])
```

# Break & Midpoint Check

---

10-minute break

- What does the dot in `math.sqrt` connect?
- Why does an unimported name raise `NameError`?
- When does a module's top-level code run?

Next: dates and times with `datetime`.

# datetime: Dates and Moments

---

- date holds a year, month and day.
- datetime also holds hours, minutes and seconds.
- today() and now() report the current moment.
- Parts are attributes: .year .month .day .hour.

```
from datetime import date, datetime

today = date.today()
print(today)      # 2026-09-27 (example)
print(today.year, today.month, today.day)

now = datetime.now()
print(now.hour, now.minute)
```

# Formatting Dates: strftime

---

- strftime turns a date into your own string layout.
- %Y, %m and %d place year, month and day.
- %A gives the full weekday name.
- Build exactly the string a report needs.

```
from datetime import date

today = date.today()
print(today.strftime("%Y-%m-%d"))
print(today.strftime("%A, %d %B %Y"))
# 2026-09-27
# Sunday, 27 September 2026 (example)
```

# Date Arithmetic: timedelta

---

- Subtracting two dates gives a timedelta in days.
- date + timedelta lands on a future date.
- .days extracts the whole-day count.
- Negative days mean the date has already passed.

```
from datetime import date, timedelta

today = date(2026, 9, 27)    # example
exam = date(2026, 11, 13)
print((exam - today).days)  # 47
print(today + timedelta(days=100))
# 2027-01-05
```

# Practice 2: Days Until a Date

---

- Open `date_check.py`.
- Read a target date as `YYYY-MM-DD`.
- Print its weekday name and the days remaining.
- Test a past date, today and a future date. 5 minutes.

Hint: `strptime(text, "%Y-%m-%d")` then subtract `date.today()`.

```
Date (YYYY-MM-DD): 2026-12-25  
Friday  
Days remaining: 89
```

```
Date (YYYY-MM-DD): 2026-09-27  
Sunday  
Days remaining: 0
```

# random: Floats and Integers

---

- `random()` returns a float from 0.0 up to, not including, 1.0.
- `uniform(a, b)` spreads floats over a wider range.
- `randint(a, b)` picks an integer; both ends included.
- Results differ on every run.

```
import random

print(random.random())      # 0.639... (varies)
print(random.uniform(1, 10)) # 8.11... (varies)
print(random.randint(1, 6))  # one die, 1..6
print(random.randint(1, 6))  # a second roll
```

# Pick, Shuffle, Sample

---

- `choice(seq)` returns one element.
- `shuffle(list)` mixes the list in place and returns `None`.
- `sample(seq, k)` picks `k` different items.
- These build on the lists from Week 4.

```
import random

names = ["Ara", "Ben", "Cho", "Dan"]
print(random.choice(names))          # one name
random.shuffle(names)
print(names)                         # new order
print(random.sample(range(1, 46), 6))
```

# Repeatability: seed

---

- The same seed produces the same sequence.
- Different seeds give different results.
- seed helps testing and reproducing a run.
- Games and lotteries rely on unseeded randomness.

```
import random

random.seed(42)
print(random.randint(1, 100)) # 82
print(random.randint(1, 100)) # 15

random.seed(42)
print(random.randint(1, 100)) # 82 again
```

# Lab 1: A number\_tools Module

---

- Finish number\_tools.py: average and count\_above.
- count\_above(numbers, limit) counts larger values.
- Finish stats\_report.py to print the report.
- Keep the \_\_main\_\_ guard demo in the module.

```
python3 stats_report.py  
Scores: [88, 92, 79, 93]  
Average: 88.0  
Above 85: 3
```

```
python3 number_tools.py  
Demo average: 5.0
```

# Lab 1 Checks

---

- Direct run: `python3 number_tools.py` prints the demo.
- Imported run: `stats_report.py` uses both tools.
- `average([])` raises `ZeroDivisionError` on an empty list.
- Handling that error gracefully is next session's topic.

```
Reports agree
Average: 88.0          352 / 4
Above 85: 3           88, 92, 93
Demo average: 5.0      2+4+6+8 over 4
```

```
Empty list
ZeroDivisionError: division by zero
```

# Lab 2: Lotto Generator

---

- Create `lotto.py`.
- Read how many games to play, from 1 to 5.
- Each game draws 6 different numbers from 1 to 45.
- Print each game on one line, sorted smallest first.

Assume integer-formatted input. One `random.sample` per game.

```
Games (1-5): 3
4 10 19 23 35 41
7 8 14 27 33 45
2 15 22 30 38 44
```

```
Games (1-5): 6
Invalid games
```

# Lab 2 Checks

---

- Input 1 prints exactly one line of six numbers.
- Input 5 prints five lines.
- Every line is sorted and has no duplicates.
- Inputs 0 and 6 print Invalid games only.

Checks for every line

Sorted: each number  $\geq$  the previous one

Unique: six different values

Range: every value between 1 and 45

Games outside 1..5

Invalid games

# Lab 3 (Take-Home): Guessing Game

---

- Create guessing.py.
- The computer picks a secret from 1 to 100.
- Each guess earns Higher, Lower or Correct!.
- Count attempts and print the count on success.

```
Guess: 50  
Higher  
Guess: 75  
Lower  
Guess: 62  
Correct! Attempts: 3
```

# How the Game Loop Works

---

- `randint(1, 100)` chooses the secret once, before the loop.
- A loop repeats until the guess matches.
- Compare with `if`, `elif` and `else`; count attempts.
- Stop the loop on a correct guess.

```
import random

secret = random.randint(1, 100)
attempts = 0
for guess in [50, 75, 62]:
    attempts += 1
    if guess < secret:
        print("Higher")
    elif guess > secret:
        print("Lower")
    else:
        print("Correct! Attempts:", attempts)
```

# NumPy: Beyond the Standard Library

---

- The standard library ships with Python itself.
- NumPy is an external package for array math.
- Arrays compute whole blocks of numbers at once.
- pandas and matplotlib, Week 7, build on it.

Colab ships NumPy preinstalled. Locally: pip install numpy.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
counts = np.array([12, 3, 45, 7])
print(prices * counts)
# [108000  75000 180000 126000]
```

# np.array: Lists Become Arrays

---

- np.array turns a list into an ndarray.
- A nested list becomes a 2D array.
- Here rows are stores, columns products.
- Building blocks are lists you already know.

```
import numpy as np

r1 = [270000, 150000, 420000, 234000]
r2 = [189000, 100000, 315000, 180000]
r3 = [108000, 75000, 180000, 126000]
sales = np.array([r1, r2, r3])
print(sales)
# [[270000 150000 420000 234000]
#  [189000 100000 315000 180000]
#  [108000 75000 180000 126000]]
```

# Arrays by Recipe

---

- `arange(1, 10, 2)` numbers a range, end excluded.
- `zeros((2, 3))` and `ones((2, 2))` prefill a shape.
- `full((4,), 0.03)` fills one repeated value.
- `linspace(0, 1, 5)` splits evenly, ends included.

```
import numpy as np

print(np.arange(1, 10, 2)) # [1 3 5 7 9]
print(np.full((4,), 0.03))
# [0.03 0.03 0.03 0.03]
print(np.linspace(0, 1, 5))
# [0.    0.25  0.5   0.75  1.   ]
```

# Random Arrays, Seeded

---

- `np.random` mirrors the `random` module.
- `randint(50, 151, size=5)` draws five integers.
- The end 151 is excluded, like `range`.
- `seed(42)` fixes the draw for everyone.

```
import numpy as np

np.random.seed(42)
print(np.random.randint(50, 151, size=5))
# [101 142  64 121 110]
print(np.random.randint(1, 7, size=3))
# [5 2 3]
```

# Reading an Array: shape and dtype

---

- shape is (rows, columns): (3, 4) here.
- ndim counts dimensions; size counts elements.
- dtype names the one shared element type.
- First questions for new data: shape? dtype?

```
import numpy as np

r1 = [270000, 150000, 420000, 234000]
r2 = [189000, 100000, 315000, 180000]
r3 = [108000, 75000, 180000, 126000]
sales = np.array([r1, r2, r3])
print(sales.shape, sales.ndim, sales.size)
# (3, 4) 2 12
print(sales.dtype) # int64
```

# One dtype per Array

---

- An array stores a single element type.
- Ints and floats together become float64.
- Numbers mixed with text become all text.
- `astype(np.float64)` converts to floats.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
print(prices.mean()) # 14000.0
print(prices.astype(np.float64))
# [ 9000. 25000.  4000. 18000.]
print(np.array([9000, "event"]))
# ['9000' 'event']
```

# Indexing and Slicing

---

- Index and slice exactly like lists.
- `prices[1:3]` starts at 1, stops before 3.
- A slice is a view of the same memory.
- Editing the view edits the original too.

Need a real copy? `prices[1:3].copy()`

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
view = prices[1:3]
view[0] = 9900
print(prices)
# [ 9000  9900  4000 18000]
```

# 2D Indexing: [row, column]

---

- One bracket pair, comma inside.
- `sales[1, 2]` picks a single cell.
- `sales[:, 1]` is a whole column.
- Each direction slices: `sales[:2, 1:3]`.

```
import numpy as np

r1 = [270000, 150000, 420000, 234000]
r2 = [189000, 100000, 315000, 180000]
sales = np.array([r1, r2])
print(sales[1, 2]) # 315000
print(sales[:, 1]) # [150000 100000]
```

# Choosing by Condition: Masks

---

- A comparison returns an array of booleans.
- Using it as an index keeps the True items.
- Combine masks with & and |, not and/or.
- Every comparison needs its parentheses.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
counts = np.array([12, 3, 45, 7])
revenue = prices * counts

print(revenue[revenue >= 100000])
# [108000 180000 126000]
mask = (revenue >= 100000) & (revenue < 150000)
print(revenue[mask]) # [108000 126000]
```

# Fancy Indexing

---

- Pass a list of positions to pick in any order.
- Duplicates and repeats are allowed.
- The result is a new array, not a view.
- Next: this lines names up with ranks.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
print(prices[[3, 0]]) # [18000 9000]
print(prices[[0, 0, 1]]) # [ 9000 9000 25000]
```

# Vectorized Math

---

- Operators apply element by element.
- `prices * 2` doubles every single value.
- A list `* 2` would repeat the whole list.
- `np.sqrt` and `np.round` map over the array.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
print(prices * 2) # [18000 50000 8000 36000]
print(prices + 1000) # [10000 26000 5000 19000]
print(np.sqrt(np.array([1, 4, 9]))) # [1. 2. 3.]
```

# Broadcasting

---

- NumPy stretches small shapes to fit big ones.
- A single number spreads across the array.
- `prices * 0.9` discounts every product.
- Trailing dimensions must match, or be 1.

```
import numpy as np

prices = np.array([9000, 25000, 4000, 18000])
print(prices * 0.9)
# [ 8100. 22500.  3600. 16200.]
promo = np.array([1000, 2000, 500, 3000])
print(prices + promo) # [10000 27000  4500 21000]
```

# Aggregation with axis

---

- sum, mean, max and min collapse an array.
- axis=0 computes down each column.
- axis=1 computes across each row.
- Check the result length: 4 products, 3 stores.

```
import numpy as np

r1 = [270000, 150000, 420000, 234000]
r2 = [189000, 100000, 315000, 180000]
r3 = [108000, 75000, 180000, 126000]
sales = np.array([r1, r2, r3])
print(sales.sum(axis=0))
# [567000 325000 915000 540000]
print(sales.sum(axis=1))
# [1074000 784000 489000]
print(sales.sum()) # 2347000
```

# reshape: Same Data, New Shape

---

- `reshape(4, 3)` regroups 12 months into quarters.
- `-1` means "take whatever remains".
- `.T` swaps rows and columns.
- The element count, size, cannot change.

```
import numpy as np

monthly = np.array([150, 162, 158, 170, 175, 188,
                    190, 197, 205, 214, 220, 232])
quarter = monthly.reshape(4, 3)
print(quarter.sum(axis=1)) # [470 533 592 666]
print(monthly.reshape(2, -1).shape) # (2, 6)
```

# Stacking: vstack and hstack

---

- vstack stacks arrays as new rows.
- hstack joins arrays as new columns.
- Pass the arrays together in one list.
- New store: vstack. New period: hstack.

```
import numpy as np

r1 = [270000, 150000, 420000, 234000]
r2 = [189000, 100000, 315000, 180000]
sales = np.array([r1, r2])
wonju = np.array([90000, 60000, 120000, 70000])

print(np.vstack([sales, wonju]).shape) # (3, 4)
print(np.hstack([sales, sales]).shape) # (2, 8)
```

# Sorting: sort and argsort

---

- `sort` returns the values, in order.
- `argsort` returns the positions, in order.
- Fancy indexing then lines names up.
- `argsort(-x)` ranks from the top instead.

```
import numpy as np

revenue = np.array([108000, 75000,
                   180000, 126000])
names = np.array(["mug", "shirt",
                  "stickers", "tumbler"])
print(names[np.argsort(revenue)])
# ['shirt' 'mug' 'tumbler' 'stickers']
```

# Guided Tour: Annual Report

---

- Reshape 12 months into four quarters.
- `axis=1` gives each quarter's total.
- `argmax` finds the best month's index.
- `argsort(-monthly)[:3]` ranks the top three.

Type along in Colab. `monthly` is in ten-thousand won.

```
import numpy as np

monthly = np.array([150, 162, 158, 170, 175, 188,
                   190, 197, 205, 214, 220, 232])
quarter = monthly.reshape(4, 3)
print(quarter.sum(axis=1)) # [470 533 592 666]
print(monthly.sum(), monthly.argmax()) # 2261 11
print(np.argsort(-monthly)[:3]) # [11 10 9]
```

# NumPy in Five Lines

---

- ndarray: many dimensions, one dtype.
- Make arrays: array, arange, zeros, linspace.
- Select data: slices, masks, fancy indexing.
- Compute: vectorization, broadcasting, axis.
- Rearrange: reshape, vstack, hstack, argsort.

Everything here returns in Week 7 with pandas.

# Common Mistakes

---

- A missing import is the first `NameError` suspect.
- `randint(1, 6)` includes 6; `range(1, 6)` does not.
- `shuffle(names)` works in place and returns `None`.
- `print(*numbers)` unpacks a list into separate values.

Before running: say where each name came from.

When debugging: check the import lines at the top first.

NumPy masks: `&` and `|` with parentheses, never `and/or`.

# Exit Quiz & Submission

---

1. What does the dot in `math.sqrt` connect?
2. Which error appears when a name was never imported?
3. What does `shuffle` return?
4. How do you draw 6 different numbers from 1 to 45?
5. What is `__name__` inside an imported module?
6. What does `sales.sum(axis=0)` collapse: rows or columns?
7. How do you rank revenue from the top in NumPy?

Submit `math_practice.py`, `date_check.py`, `number_tools.py`, `stats_report.py` and `lotto.py`. `guessing.py` is due next session.  
Record at least three test cases in comments for each program.

# Next Session: Exceptions

---

- See why `int("abc")` raises `ValueError`.
- Keep programs alive with `try` and `except`.
- Choose a suitable reaction for each failure.

Preparation: find one input that breaks a Week 4 program.

File input and output, then classes, follow in Week 6.